

Вариант для продвинутого

Задача 1

Рассмотрим следующую программу:

```
#include <iostream>

int main() {
    for (int i = 0; i < 300; ++i) {
        std::cout << i << ' ' << i*12345678 << std::endl;
    }
}
```

После компиляции на g++ с параметром -O2 и запуска выяснилось, что цикл в этой программе работает бесконечно. Объясните подробно: как такое могло произойти? Что именно сделал компилятор и почему? Какую роль сыграл параметр -O2 при компиляции?

Задача 2

Рассмотрим следующую программу:

```
#include <iostream>

int f() {
    std::cout << 1;
    return 1;
}

int g() {
    std::cout << 2;
    return 2;
}

int h() {
    std::cout << 3;
    return 3;
}

int main() {
    std::cout << (f() + g() * h());
}
```

Согласно Стандарту, какие варианты вывода этой программы возможны? Объясните подробно: каким образом возможен каждый из них и почему невозможны другие?

Задача 3

Рассмотрим следующую программу:

```
#include <iostream>

int main() {
```

```

const int N = 10'000'000;
/*static*/ int a[N];
for (int i = 0; i < N; ++i) {
    a[i] = i;
}
}

```

После компиляции и запуска этой программы обнаружилось, что она вызывает Segmentation fault. Однако, если раскомментировать слово static, ошибка Segmentation fault исчезает. Объясните подробно: почему в первом случае происходит эта ошибка и почему она исчезает, если добавить слово static?

Задача 4

Рассмотрим следующую программу:

```

#include <iostream>

int& f(int x) {
    if (x > 0) {
        return x;
    } else {
        return x * x;
    }
}

int main() {
    std::cout << f(5);
}

```

Оказалось, что эта программа не компилируется. Объясните подробно: что именно вызывает ошибку компиляции? Почему так происходит?

Задача 5

Рассмотрим следующую реализацию метода [] для класса String:

```

class String {
private:
    size_t sz;
    char* str;
public:
    char& operator[](size_t i) const {
        return str[i];
    }
    // ... other methods
};

```

В чем проблема такой реализации? Скомпилируется ли она, будет ли правильно работать? Как надо исправить, чтобы всё стало корректно? Объясните подробно.

Задача 6

Рассмотрим следующий код:

```
struct Segment {
    double begin;
    double end;
};

void processSegments(std::vector<Segment>& segments, bool invert = false) {
    for (Segment& s: segments) {
        if (!invert) {
            do_some_operations(s.begin, s.end);
            do_first_operation(s.begin);
            do_second_operation(s.end);
        } else {
            do_some_operations(s.end, s.begin);
            do_first_operation(s.end);
            do_second_operation(s.begin);
        }
    }
}
```

Используя указатели на члены, упростите код функции processSegments так, чтобы проверку условия invert нужно было сделать всего $O(1)$ раз (и чтобы весь for при этом не загонять внутрь if).

Задача 7

Рассмотрим код:

```
struct Complex {
    double re = 0.0;
    double im = 0.0;
    Complex(double x = 0.0, double y = 0.0): re(x), im(y) {}
    Complex(const Complex&) {
        std::cout << "A";
    }

    Complex& operator+=(const Complex& z) {
        re += z.re;
        im += z.im;
        return *this;
    }
}

Complex operator+(const Complex& a, const Complex& b) {
    Complex copy = a;
    copy += b;
    return copy;
}
```

```
int main() {  
    Complex c(2.0);  
    Complex d(3.0, 1.0);  
    Complex sum = c + d;  
}
```

При компиляции и запуске этого кода выясняется, что конструктор копирования `Complex` вызвался 1 раз (на экран выводится одна буква `A`). Однако, если вместо двух выделенных строчек написать одну `"return sору += b;"`, то на экран выведется две буквы `A`. Объясните как можно подробнее, в чем разница и почему так происходит?

Задача 8

Рассмотрим код:

```

#include <iostream>

class Granny {
private:
    int g = 0;
    friend class Son;
};

class Mom: protected Granny {
private:
    int m = 1;
    void f() {
        std::cout << g; // 1
    }
};

class Son: private Mom {
protected:
    int s = 2;
    friend int main();
    void f() {
        std::cout << m << g; // 2
    }
};

int main() {
    Son s;
    std::cout << s.s << s.m << s.g; // 3
}

```

При компиляции оказалось, что в строке 1 нельзя обратиться к `g`, в строке 2 нельзя обратиться к `m` (но можно к `g`), а в строке 3 нельзя обратиться ни к `m`, ни к `g` (но можно к `s`).

Про каждый из 6 случаев обращения к полям объясните, почему в данном случае происходит (или, соответственно, не происходит) ошибка компиляции: чем именно регулируется доступ в каждом случае?

Задача 9

Рассмотрим код:

```
#include <iostream>

struct Granny {
    int g;
};

struct Mom: public virtual Granny {
    int m;
};

struct Dad: public virtual Granny {
    int d;
};

struct Son: public Mom, public Dad {
    int s;
};

int main() {
    Son s;
    std::cout << sizeof(s);
}
```

После запуска оказалось, что на экран выводится число 40. Объясните как можно подробнее, почему размер объекта s равен этому числу (из чего состоит этот объект и сколько байт занимают его части).

Задача 10

Рассмотрим код:

```

#include <iostream>

struct Granny {
    int g = 0;
    virtual void f() {
        std::cout << g;
    }
};

struct Mom: private Granny {};

struct Dad: public Granny {};

struct Son: public Mom, private Dad {};

int main() {
    Son s;
    s.f(); // 1

    Mom& m = s;
    m.f(); // 2
}

```

Произойдет ли ошибка компиляции в строке 1? А в строке 2? В каждом случае как можно подробнее объясните, почему. (Обратите внимание, что функция f виртуальная, а некоторые наследования приватные; это выделено полужирным, чтобы вы случайно не упустили это из виду.)

Задача 11

Рассмотрим программу:

```

#include <iostream>

template <typename T, typename U>
void f(T x, U y) {
    std::cout << 1 << '\n';
}

template <>
void f(int x, int y) {
    std::cout << 2 << '\n';
}

template <typename T>
void f(T x, T y) {
    std::cout << 3 << '\n';
}

int main() {
    f(0, 0);
}

```

Если ее скомпилировать и запустить, на экран выведется 2. Однако, если поменять местами определения двух последних функций, то поведение поменяется и на экран будет выводиться 3. Объясните: почему так происходит? Каким образом и по каким правилам компилятор выбирает версию функции для вызова в каждом случае?

Задача 12

Рассмотрим следующий код:


```

template <int N>
struct Danger {
    int a[N];
};

template <typename T, int N>
struct S {
    /*virtual*/ void f() {
        Danger<N> d;
    }

    void g() {
        Danger<-1> d;
    }
};

//template void S<int, -1>::g();

int main() {
    S<int, -1> s;
}

```

Если попытаться его скомпилировать, ошибки не возникнет.

Однако если раскомментировать строку, выделенную полужирным, возникает ошибка компиляции. Также ошибка компиляции возникает, если раскомментировать слово `virtual`. Объясните как можно подробнее: почему возникает ошибка компиляции в каждом из этих случаев, хотя изначально ее не было?

Задача 13

На лекции мы рассматривали следующую (неудачную) реализацию метафункции `is_homogeneous`. Она должна была проверять, все ли переданные типы одинаковы.

```

template <typename First, typename Second, typename... Args>
struct is_homogeneous {
    static const bool value = std::is_same_v<First, Second>
        && is_homogeneous<Second, Args...>::value;
};

template <typename T>
struct is_homogeneous<T, T> {
    static const bool value = true;
};

```

При попытке вызвать `is_homogeneous<int, char, bool>::value` мы получаем ошибку компиляции со словами `wrong number of template arguments (1, should be at least 2)`. Объясните, почему происходит такая ошибка, и как надо исправить реализацию, чтобы все работало корректно.

Задача 14

Реализуйте шаблонную структуру `isPerfectSquare<size_t N>`. В ней должно быть поле `value`, равное `true` в случае, если число `N` является квадратом некоторого натурального числа, и `false` иначе. (Ноль - тоже натуральное число, как вы знаете из курса мат.логики :))

Асимптотика решения роли не играет. При реализации запрещено пользоваться специальными возможностями C++11 (а именно, словами `constexpr` и `static_assert`).

Основной поток - вариант 1

Задача 1. Про SE в тернарном операторе

Рассмотрим следующий код:

```
#include <iostream>

int main() {
    int x = -1;
    (x > 0 ? ++x : x++) = 0;
}
```

Оказалось, что этот код вызывает ошибку компиляции. Объясните как можно подробнее, из-за чего возникает ошибка компиляции в данном случае.

Исчезнет ли ошибка компиляции, если изначально переменную x проинициализировать числом 1? В любом случае объясните ответ.

Задача 2. Про unspecified

Рассмотрим следующую программу:

```
#include <iostream>

int f() {
    std::cout << 1;
    return 1;
}

int g() {
    std::cout << 2;
    return 2;
}

int h() {
    std::cout << 3;
    return 3;
}

int main() {
    std::cout << (f() + g() * h());
}
```

Если скомпилировать и запустить эту программу, на экран выводится 1237. Согласно Стандарту, возможны ли еще какие-то варианты вывода этой программы? Объясните подробно: каким образом возможен каждый из вариантов и почему невозможны другие?

Задача 3. Про ссылки

Рассмотрим следующую программу:

```
#include <iostream>

void inc_value(int value) { ++value; }
void inc_ref(int& ref) { ++ref; }

void assign_value_from_ref(int value, int& ref) { value = ref; }
void assign_ref_from_value(int& ref, int value) { ref = value; }

int main() {
    int x = 1;
    int y = 2;

    int& a = x;
    int& b = y;

    inc_value(x);
    inc_ref(a);

    inc_value(b);
    inc_ref(y);

    assign_value_from_ref(x, a);
    assign_ref_from_value(b, y);

    std::cout << x << y << a << b;
}
```

Что выведется на экран, если ее скомпилировать и запустить? Приведите развернутое объяснение (что происходит с переменными во время вызова каждой из функций и что получается в итоге работы программы).

Задача 4

Рассмотрим следующую программу:

```
#include <iostream>
class A {
public:
    int x;
    int& y;

    A(int xx, int yy) { x = xx; y = yy; }
    A(const A&) = default;
};
int main() {
    A a(1, 2);
    A b = a;
    std::cout << a.x << a.y;
}
```

Какая ошибка возникнет при попытке скомпилировать этот код? Объясните, в чем причина ошибки. Исправьте конструкторы класса A так, чтобы код начал компилироваться.

Задача 5

Рассмотрим следующую программу:

```
#include <iostream>
class String {
private:
    int sz;
    char* data;
public:
    String(size_t sz) {
        this->sz = sz;
        data = new char[sz];
    }
    ~String() {
        delete[] data;
    }
};

int main() {
    String s1(10);
    String s2 = s1;
    s1 = s2;
}
```

Какая проблема возникнет при ее запуске? Как нужно дополнить реализацию класса String, чтобы избавиться от этой проблемы? Допишите недостающий код.

Задача 6

Рассмотрим код:

```
#include <iostream>
#include <set>

int main() {
    std::set<int> s;
    s.insert(5);
    s.insert(-3);
    s.insert(2);
    for (int x: s) {
        std::cout << x;
    }
}
```

Если запустить эту программу, на экран выводится -325. (Что логично, так как set сортирует элементы по возрастанию.) Как надо изменить объявление s и что дописать перед main, чтобы элементы в этом set сортировались по возрастанию **модуля**?

Напишите соответствующий недостающий код и поясните, как он будет работать.

Задача 7.

Рассмотрим программу:

```
#include <iostream>

class A {
public:
    A() { std::cout << 'a'; }
    ~A() { std::cout << 'A'; }
};

class C {
public:
    C() { std::cout << 'c'; }
    ~C() { std::cout << 'C'; }
};

class B: public A {
private:
    C c;
public:
    B() { std::cout << 'b'; }
    ~B() { std::cout << 'B'; }
};

int main() {
    B b;
    A a = b;
}
```

Что выведется на экран при ее запуске? Объясните свой ответ (в каком порядке будут вызываться конструкторы и деструкторы и почему).

Задача 8

Рассмотрим следующий код:

```

#include <iostream>

class Granny {
protected:
    int g = 0;
    friend int main();
};

class Mom: private Granny {
private:
    int m = 1;
    void f() {
        std::cout << g; // 1
    }
    friend class Son;
};

class Son: public Mom {
protected:
    int s = 2;
    void f() {
        std::cout << m << g; // 2
    }
};

int main() {
    Son s;
    std::cout << s.s << s.m << s.g; // 3
}

```

В каких из выделенных строк (1, 2 и 3) возникнут ошибки компиляции? Как можно подробнее поясните свой ответ: какие переменные будут доступны из этих мест, а какие недоступны, и почему?

Задача 9.

Рассмотрим следующий код:


```

#include <iostream>

struct Granny {
    int g = 0;
};

class Mom: public Granny {
    int m = 1;
};

class Dad: private Granny {
    int d = 2;
};

class Son: private Mom, public Dad {
    int s = 3;
};

int main() {
    Son s;

    std::cout << s.g; // 1
    std::cout << static_cast<Mom*>(s).m << static_cast<Mom*>(s).g; // 2
    std::cout << static_cast<Dad*>(s).d << static_cast<Dad*>(s).g; // 3
}

```

Ответьте на вопросы:

- 1) В каких из строк 1, 2, 3 возникнут ошибки компиляции и почему? Как можно подробнее поясните свой ответ.
- 2) Что изменится, если все `static_cast` заменить на `reinterpret_cast`? Исчезнут ли старые ошибки компиляции, появятся ли какие-то другие ошибки вместо старых? Поясните свой ответ.

Задача 10

Рассмотрим код:

```

#include <iostream>
struct Base {
    virtual void f() const { std::cout << 1; }
};
struct Derived: public Base {
    virtual void f() /*override*/ { std::cout << 2; }
};
int main() {
    Derived d;
    d.f();
    Base& b = d;
    b.f();
}

```

Ответьте на вопросы:

- 1) Скомпилируется ли этот код? Если нет, то почему, а если да, то что выведется на экран и почему?
- 2) Что изменится, если раскомментировать слово `override`? Поясните свой ответ как можно подробнее.

Задача 11

Рассмотрим код:

```

struct Base {
    ~Base() = 0;
};

struct Derived: public Base {
    int* dp = new int();

    ~Derived() {
        std::cout << "Derived\n";
        delete dp;
    }
};

int main() {
    Base* b = new Derived();
    delete b;
}

```

При попытке скомпилировать и запустить эту программу возникает следующая ошибка:

```
/tmp/ccyDyDBH.o: In function `Derived::Derived()':
test_virt_destructor.cpp:(.text._ZN7DerivedC2Ev[_ZN7DerivedC5Ev]+0x4c): undefined
reference to `Base::~~Base()'
collect2: error: ld returned 1 exit status
```

Ответьте на вопросы:

- 1) Что нужно дописать в код, чтобы эта ошибка исчезла? Поясните свой ответ.
- 2) Что изменится, если в исходном коде удалить выделенную строку (то есть сделать тело Base пустым)? Скомпилируется ли код после этого и будет ли он работать корректно? Поясните свой ответ.

Задача 12.

Рассмотрим программу:

```
#include <iostream>

template <typename T, typename U>
void f(T x, U y) {
    std::cout << 1 << '\n';
}

template <>
void f(int x, int y) {
    std::cout << 2 << '\n';
}

template <typename T>
void f(T x, T y) {
    std::cout << 3 << '\n';
}

int main() {
    f(0, 0);
}
```

Известно, что если ее скомпилировать и запустить, то на экран выведется 3. Однако, если поменять местами определения двух последних функций, то поведение поменяется и на экран будет выводиться 2. Объясните: почему так происходит? Каким образом и по каким правилам компилятор выбирает версию функции для вызова в каждом случае?

Задача 13

Рассмотрим код:

```
#include <iostream>

template <typename T, size_t N>
struct SimpleArray {
    T a[N];
};
```

```
int main() {
    size_t n = 5;
    SimpleArray<int, n> a;

    for (int i = 0; i < n; ++i) {
        a.a[i] = i;
    }
}
```

Оказалось, что данный код вызывает ошибку компиляции. Объясните как можно подробнее: из-за чего возникает ошибка? Как можно исправить код, чтобы она исчезла?

Задача 14

Рассмотрим код:

```
template <typename T>
struct S {
    using x = T;
};

template <>
struct S<int> {
    static int x;
};

template <typename T>
void f() {
    S<T>::x * a;
}

int main() {}
```

Оказалось, что выделенная строка вызывает ошибку компиляции. Ответьте на вопросы:

- 1) В чем заключается ошибка компиляции для данного кода? Поясните как можно подробнее.
- 2) Что можно дописать в код (не добавляя в него комментариев и макросов), чтобы ошибка компиляции исчезла? Поясните как можно подробнее.

Задача 15. Реализуйте `remove_all_extents`

Реализуйте шаблонную метафункцию `remove_all_extents<T>`. Это должна быть структурой, в которой присутствует внутренний тип `type`, равный исходному типу, если он не был массивом, и равный `T`, если `T` имел вид `T[][...]` (си-стайл массив с произвольным количеством измерений). Другими словами, ваша структура должна работать аналогично `std::remove_extent`, но удалять не одно измерение, а сразу все.

Внимание. Когда будете вводить код в поле для ответа, может возникнуть проблема с символами `<` `>` (данный сайт воспринимает их как спецсимволы, и то, что вы напечатаете внутри них, не будет отображаться при проверке). Возможные пути решения этой проблемы: 1) написать решение в текстовом редакторе и вставить скрин; 2) экранировать символы `<` символами `$`, тогда они не будут восприниматься как спецсимволы.

Вариант 2

Задача 1. Про UB в цикле

Рассмотрим следующую программу:

```
#include <iostream>

int main() {
    for (int i = 0; i < 300; ++i) {
        std::cout << i << ' ' << i*12345678 << std::endl;
    }
}
```

После компиляции на g++ с параметром -O2 и запуска выяснилось, что цикл в этой программе работает бесконечно. Объясните подробно: как такое могло произойти? Что именно сделал компилятор и почему? Какую роль сыграл параметр -O2 при компиляции?

Задача 2. Битая ссылка

Рассмотрим следующий код.

```
#include <iostream>
#include <string>

using std::string;

const string& f(const string& s) {
    return s;
}

int main() {
    string s = f("abc");
    std::cout << s;
}
```

Какая проблема присутствует в этом коде? Объясните подробно, из-за чего она возникает. Как надо поправить код, чтобы этой проблемы не стало?

Задача 3. Битовое представление дабла

Рассмотрим код:

```
#include <iostream>
```

```
int main() {
    double pi = 3.14;

    // ...
}
```

Напишите код вместо `// ...`, который бы позволил увидеть битовое представление переменной `pi` в памяти. Подсказка: вам могут пригодиться побитовые арифметические операторы, а также оператор `reinterpret_cast`.

Внимание. Когда будете вводить код в поле для ответа, может возникнуть проблема с угловыми скобками, т.е. символами “больше” и “меньше” (данный сайт воспринимает их как спецсимволы, и то, что вы напечатаете внутри них, не будет отображаться при проверке). Возможные пути решения этой проблемы: 1) написать решение в текстовом редакторе и вставить скрин; 2) экранировать эти символы символами `$`, тогда они не будут восприниматься как спецсимволы.

Задача 4.

Рассмотрим следующий код.

```
#include <iostream>
class A {
public:
    int x;
    int& y;

    A() = default;
    A& operator=(const A&) = default;
    A(const A&) = default;
};
int main() {
    A a;
    A b = a;
    a = b;
    std::cout << a.x << a.y;
}
```

Оказалось, что этот код вызывает ошибку компиляции. Поясните подробно: из-за чего возникает ошибка? Предложите вариант определения конструкторов и оператора присваивания для класса `A`, при котором бы ошибок компиляции не было.

Задача 5

Рассмотрим код:

```

#include <iostream>

class C {
private:
    int field = 0;

    C() {
        field = 1;
        std::cout << "C " << field << '\n';
    }
public:
    C(int x) {
        C(); // !!!
        std::cout << "C " << field << '\n';
        field = x;
    }

    ~C() {
        std::cout << "~C " << field << '\n';
    }
};

int main() {
    C c(2);
}

```

Что выведется на экран при запуске этого кода? Поясните подробно, что будет происходить. Обратите внимание на строчку, выделенную полужирным. Что вы можете сказать о ее корректности? Как правильно следовало бы вызывать конструктор C() из другого конструктора?

Задача 6. [] у String

Рассмотрим следующую реализацию метода [] для класса String:

```

class String {
private:
    size_t sz;
    char* str;
public:
    char& operator[](size_t i) const {
        return str[i];
    }
    // ... other methods
};

```

В чем проблема такой реализации? Скомпилируется ли она, будет ли правильно работать? Как надо исправить, чтобы всё стало корректно? Объясните подробно.

Задача 7. explicit и перегрузка приведения типа

Рассмотрим код:

```
struct UserId {
    int id = 0;

    /*explicit*/ UserId(int id): id(id) {} // 1

    /*explicit*/ operator int() { // 2
        return id;
    }
};

int operator+(UserId a, UserId b) {
    return a.id + b.id;
}

int main() {
    UserId id = UserId(5);
    std::cout << id + 1;
}
```

Оказалось, что этот код вызывает ошибку компиляции в строке, выделенной полужирным. Ответьте на вопросы:

- 1) Из-за чего происходит ошибка компиляции в данном коде?
- 2) Что изменится, если раскомментировать слово `explicit` в строке 1? Объясните подробно.
- 3) Что изменится, если раскомментировать оба слова `explicit`? Объясните подробно.

Задача 8. Доступы при наследовании

Рассмотрим следующий код:

```
#include <iostream>

class Granny {
protected:
    int g = 0;
    friend int main();
};

class Mom: protected Granny {
private:
    int m = 1;
    void f() {
        std::cout << g; // 1
    }
    friend class Son;
};

class Son: protected Mom {
public:
    int s = 2;
    void f() {
```



```

        std::cout << m << g; // 2
    }
};

int main() {
    Son s;
    std::cout << s.s << s.m << s.g; // 3
}

```

В каких из выделенных строк (1, 2 и 3) возникнут ошибки компиляции? Как можно подробнее поясните свой ответ: какие переменные будут доступны из этих мест, а какие недоступны, и почему?

Задача 9

Рассмотрим код:

```

#include <iostream>
struct Base {
    virtual void f() { std::cout << 1; }
};
struct Derived: public Base {
    virtual void f() const { std::cout << 2; }
};
int main() {
    Derived d;
    d.f();
    static_cast<Base&>(d).f();
}

```

Скомпилируется ли он, и если нет, то по какой причине? Объясните как можно подробнее. Если скомпилируется, то что выведется на экран при его запуске? Объясните подробно, какая версия f выберется в первом случае и какая - во втором, и почему так произойдет.

Задача 10

Рассмотрим код:

```

#include <iostream>

struct Granny {
    /*virtual*/ void foo() { std::cout << "Granny\n"; }
};

struct Mother: public Granny {
    void foo() { std::cout << "Mother\n"; }
};

struct Father: public Granny {
    void foo() { std::cout << "Father\n"; }
}

```

```
};

struct Son: public Mother, public Father {};

int main() {
    Son s;
    Mother& m = s;
    dynamic_cast<Father&>(m).foo(); // 1
    dynamic_cast<Son&>(m).foo(); // 2
}
```

Оказалось, что данный код не компилируется: ошибки компиляции возникают в строках 1 и 2. Ответьте на вопросы:

- 1) Почему возникают ошибки компиляции в данном коде?
- 2) Поменяется ли что-нибудь, если раскомментировать слово `virtual`? Исчезнут ли ошибки, появятся ли другие вместо них? Поясните подробно.
- 3) Поменяется ли что-нибудь, если оставить закоментированным `virtual`, но вместо `dynamic_cast` везде написать `static_cast`? Поясните подробно.

Задача 11

Рассмотрим код:

```
#include <iostream>

struct Base {
    int a = 0;
    int b = 1;
    void f() {}
    virtual void g() {}
};

struct Derived: public Base {
    int a = 2;
    int c = 3;
    virtual void f() {}
    void g() override {}
    void h() {}
};

int main() {
    std::cout << sizeof(Derived);
}
```

Оказалось, что этот код выводит на экран число 24. Объясните как можно подробнее, почему размер объекта Derived будет равен этому числу (из чего состоит этот объект и сколько байт занимают его части).

Задача 12. Выбор версии перегрузки

Рассмотрим код:

```
template <typename T, typename U>
void f(T x, U y) {
    std::cout << 1 << '\n';
}

void f(int x, int y) {
    std::cout << 2 << '\n';
}

template <typename T>
void f(T x, T y) {
    std::cout << 3 << '\n';
}

int main() {
    f(0, 0);
    f(0.0, 0.0);
    f(0, 0.0);
    f<int>(0, 0);
}
```

Что выведется на экран при его запуске? Объясните подробно, какая версия f выберется в каждом случае и почему. Если вы считаете, что возникнет ошибка компиляции, также поясните, почему она возникнет.

Задача 13. Шаблонные параметры шаблона

Рассмотрим код:

```
template <typename T, typename Container>
struct Stack {
    Container c;
};

int main() {
    Stack<int, std::vector> s;
}
```

Оказалось, что выделенная полужирным строка вызывает ошибку компиляции. Объясните: из-за чего возникает ошибка? Как надо поправить объявление и определение структуры Stack, чтобы ошибка компиляции исчезла?

Задача 14. про typename - ок

Рассмотрим код:

```
template <typename T>
struct S {
    using x = T;
};

template <>
struct S<int> {
    static int x;
};

template <typename T>
void f() {
    S<T>::x * a;
}

int main() {}
```

Оказалось, что выделенная строка вызывает ошибку компиляции. Ответьте на вопросы:

- 1) В чем заключается ошибка компиляции для данного кода? Поясните как можно подробнее.
- 2) Что можно дописать в код (не добавляя в него комментариев и макросов), чтобы ошибка компиляции исчезла? Поясните как можно подробнее.

Задача 15. Реализуйте print

Реализуйте шаблонную функцию print с переменным количеством шаблонных аргументов, которая принимала бы произвольное число аргументов произвольных типов и выводила бы их в std::cout через

пробел. Приведите пример использования вашей функции `print` из `main`. Постарайтесь написать законченную программу, так, чтобы ваш код компилировался.

Внимание. Когда будете вводить код в поле для ответа, может возникнуть проблема с угловыми скобками, т.е. символами “больше” и “меньше” (данный сайт воспринимает их как спецсимволы, и то, что вы напечатаете внутри них, не будет отображаться при проверке). Возможные пути решения этой проблемы: 1) написать решение в текстовом редакторе и вставить скрин; 2) экранировать эти символы символами `$`, тогда они не будут восприниматься как спецсимволы.

Правильные решения

Утренняя волна

Задача 1.

СЕ возникает из-за того, что выражение `(x > 0 ? ++x : x++)` является `rvalue`, а ему нельзя присваивать.

Даже если изначально проинициализировать `x=1`, СЕ не исчезнет. Хотя `++x` сам по себе является `lvalue`, компилятор обязан принять решение относительно всего выражения `(x > 0 ? ++x : x++)`, чем оно является. Поскольку одна из частей является `rvalue`, то и все выражение целиком является `rvalue`, значение `x` роли не играет.

Задача 2.

Возможны 6 вариантов: цифры 123 могут образовывать любую перестановку, на конце обязательно будет 7.

Хоть приоритет оператора `*` и выше, чем `+`, нет гарантий относительно того, в каком порядке будут вычисляться значения самих аргументов. У каждого из операторов может сначала вычислиться левый операнд, потом правый, а может наоборот. Отсюда `unspecified behaviour`.

NB: за ответ “возможны всего 4 варианта” баллы не снижаем. Это детали стандарта, в которые мы не лезем. Главное - наличие пояснения, что порядок 123 не определен, но 7 обязательно будет в конце.

Задача 3.

Выведется 2323. Функции `inc_value` и `assign_value_from_ref` никак не меняют переданные параметры, так что для целей задачи можно считать, что этих функций просто нет.

`inc_ref(a)` увеличивает `a` (то есть `x`) на 1.

`inc_ref(b)` увеличивает `b` (то есть `y`) на 1.

`assign_ref_from_value(b, y)` ничего не меняет, поскольку `b` и так равно `y` (и не просто равно ему, а это он и есть).

В итоге получаем вывод 23, после чего еще раз то же самое 23.

Задача 4.

Тут проблема в том, что поле `int&` у ничем не проинициализировано перед входом в конструктор. Надо написать список инициализации вместо присваивания полей: `A(int xx, int yy): x(xx), y(yy) {}`. Конструктор копирования менять не нужно, он сам нормально сгенерится. Правда, теперь есть новая проблема - UB из-за того, что поле `y` будет висячей ссылкой, ну да вопрос уж не об этом.

Задача 5

Ну тут все очевидно, нужен нетривиальный копи-конструктор и оператор присваивания. Проверить, что правильно объяснили проблему и правильно их реализовали.

Задача 6

Тут надо написать перед мэйном класс-компаратор, определить в нем оператор () как надо, а в самом сете указать его шаблонным параметром.

Задача 7

Будет `asbABCA`.

Сначала создается объект В. При этом сначала создается родитель (a), потом поле наследника (c), потом сам наследник (b).

Потом происходит срезка и копируется родительская часть. Ничего при этом не выводится.

Потом уничтожается объект А (который был копией исходного).

Потом уничтожается исходный объект: сначала деструктор наследника (В), потом поля наследника (С) и в конце родитель (А).

Задача 8.

В строке 1 ошибки не будет: Mom может обратиться к полю своего непосредственного родителя, т.к. оно `protected`.

В строке 2 тоже ошибки не будет. Son может обратиться к полю `m` своего непосредственного родителя, хоть оно и `private`, но там написано `friend class Son`. К полю `g` он может обратиться, поскольку хоть Granny и приватный родитель, но мама к ней пускает (сын - друг для мамы), а у бабушки поле уже `protected`, а не `private`.

В строке 3 будет СЕ во всех трех случаях. `s.s` защищенное поле, `s.m` приватное поле, а `s.g` недоступно, потому что хоть у бабушки и написано `friend int main`, мама к ней не пускает (наследование приватное).

Задача 9

`s.g` вызывает неоднозначность из-за ромбовидного наследования (важный момент: приватность тут не играет роли, ошибка именно по причине неоднозначности, т.к. доступы проверяются после выбора версии).

`static_cast<Mom&>` вызывает ошибку доступа, т.к. наследование приватное.

`static_cast<Dad&>` ошибку не вызывает, но обращение к полям все равно вызывает СЕ, т.к. поля приватные.

Если заменить на `reinterpret`, то пропадет ошибка из-за запрещенного каста. Однако обращение к полям все равно нигде работать не будет, т.к. они по-прежнему все приватные.

Задача 10

В первом случае выведется 21. Сначала 2 - очевидно. Потом 1 - потому что `f` в классе `Derived` реально не оверрайдит `f` из класса `Base` (сигнатура не совпадает), поэтому выбирается версия для `Base`.

Если раскомментировать `override`, то будет СЕ - как раз потому, что `f` обещала оверрайдить, а на самом деле не оверрайдит.

Задача 11

- 1) Поскольку деструктор родителя вызывается после деструктора наследника, а деструктор родителя не определен, возникает ошибка линковщика. Надо дописать определение деструктора. Сделать это надо вне тела класса: `Base::~~Base() {}`
- 2) Если удалить виртуальный деструктор, то для переменной `b` не будет вызван деструктор `Derived`, а будет вызван лишь неявно сгенерированный пустой деструктор `Base`. Как следствие - `delete dp` не будет сделано, это потенциальная утечка памяти.

Задача 12

Компилятор сначала выбирает подходящую версию перегрузки, а затем выбирает подходящую специализацию этой версии, если таковая имеется. В первом случае `f(T, T)` выигрывает у `f(T, U)` и

поэтому выбирается $f(T, T)$, а специализаций у нее нет. Во втором случае $f(int, int)$ становится специализацией $f(T, T)$, а не $f(T, U)$, поэтому выбирается эта специализация, а не оригинальная версия $f(T, T)$.

Задача 13

Параметром шаблона не может быть неконстантная переменная, т.к. компилятор должен в момент компиляции во все шаблоны подставить фиксированные значения переменных, а он не может так поступить с неконстантной переменной. Все заработает, если просто объявить `n` как `const size_t`.

Задача 14

По стандарту компилятор парсит строку `S<T>::x * a;` как выражение, а не как объявление. Он не может парсить ее по-разному в зависимости от `T`, так как он должен еще до шаблонной подстановки уметь проверять корректность синтаксиса, в частности, отличать объявления от выражений. Поэтому возникает ошибка “`a` - необъявленная переменная”. Можно исправить ошибку двумя способами: 1) дописать `typename` перед этой строкой, и тогда он будет парсить это как объявление; 2) объявить переменную `a` где-то перед этой строкой, тогда он успешно распарсит это как выражение.

Задача 15

Например, так:

```
template<typename T>
struct remove_all_extents {
    using type = T;
};

template<typename T>
struct remove_all_extents<T[]> {
    using type = remove_all_extents<T>;
};
```

Вечерняя волна

Задача 1

Тут при `-O2` срабатывает следующая оптимизация: компилятор предполагает, что `i*12345678` никогда не переполняется, поэтому `i<=174`, поэтому `i<300` всегда верно, а значит, можно считать условие цикла тождественно равным `true`. Это пример UB в коде, которое возникло из-за того, что переменная типа `int` была переполнена. (Этот пример разбирался на лекциях)

Задача 2

Тут UB. “`abc`” - это `rvalue`, оно связывается с параметром функции `const string& s` и этот объект существует до тех пор, пока этот параметр не выйдет из области видимости. Но из функции возвращается тоже `const string&`, а значит, мы получаем из функции ссылку на объект, который уничтожается в момент выхода из функции. То есть мы получаем `dangling reference`.

Чтобы избежать UB, можно, например, возвращать из функции объект по значению, а не по ссылке.

Задача 3

Ну, например, так (можно и как-нибудь по-другому). Главное - правильно использовать `reinterpret_cast` (кастовать к ссылке или к указателю, а не просто к `uint64`).

```
double pi = 3.14;
unsigned long long x = reinterpret_cast<unsigned long long>(pi);
for (int i = 0; i < 16; ++i) {
    std::cout << (x & (1 << i) ? 1 : 0);
```

}

Задача 4

Тут проблема с инициализацией поля-ссылки. Для такого класса не получится сгенерить по умолчанию ни A(), ни оператор присваивания. Можно написать, например, так:

```
A(): x(0), y(x) {}
```

Ну или еще как-то, лишь бы проинициализировать y перед входом в конструктор каким-то lvalue.

С оператором присваивания вообще трудно что-то придумать осмысленное, но можно и бессмысленное. Например, просто ничего не делать:

```
A& operator=(const A&) { return *this; }
```

Конструктор копирования можно не исправлять, он и по умолчанию сгенерится нормально.

Задача 5

Тут выведется:

C 1

~C 1

C 0

~C 2

Из конструктора C(2) вызывается конструктор C(), создавая локальный объект со значением field=1, и этот объект сразу же уничтожается. После чего уже дорабатывает конструктор C(2) и потом деструктор. Тут намек на делегирующие конструкторы. Если бы мы хотели из конструктора C(2) вызвать сначала C() для того же объекта, то надо было бы писать C(int x): C() {...}, а не вызывать C() прямо в теле нашего конструктора.

Задача 6

Такая реализация скомпилируется, но будет работать странно: получится, что можно менять константные строки (делать присваивания в духе s[i] = 'a'). Потому что в константном методе константными считаются сами поля, а именно в нашем случае указатель str, но не то, что лежит под ним. То есть char* const str, а не const char* str.

Чтобы работало как следует, надо сделать две перегрузки [], одну const и возвращающую const char& (или просто char), а другую без const и возвращающую char&.

Задача 7

Тут СЕ происходит из-за неоднозначности: непонятно, приводить ли 1 к UserId или id к int, прежде чем выполнять сложение. Эти две версии + одинаково хорошо подходят.

Если раскомментировать первое explicit, то первая из этих версий отпадет и ошибка исчезнет, сложение будет единственным образом пониматься как сложение интов.

Если же раскомментировать оба explicit, то снова будет СЕ, но на этот раз уже из-за отсутствия подходящих конверсий: ни одна из версий + неприменима, т.к. оба каста теперь не могут выполняться неявно.

Задача 8

В строке 1 ошибки компиляции не будет: мама может обращаться к полям своего непосредственного родителя, т.к. они protected.

В строке 2 тоже ошибки не будет. К полю m можно обратиться, хоть оно и приватное, т.к. у мамы написано friend class Son, а к полю g можно обратиться, т.к. бабушка - protected родитель (и поэтому доступна нам как наследнику), и поля у нее тоже protected.

В строке 3 будет СЕ. s.s ошибки не вызовет; s.m недоступно, т.к. мама - protected родитель и мы не можем из main к ней обратиться (да и тем более поля у нее private), а s.g недоступно потому, что наследование protected и private соответственно. friend int main в бабушке не спасает, т.к. сама бабушка приватная.

Задача 9

Выведется 21.

В первом случае 2, потому что поиск имен находит имя f в классе Derived и не идет искать другие

версии в Base.

Во втором случае 1, потому что константная `f` и неконстантная `f` - это две разных функции, и константная `f` не оверрайдит неконстантную версию.

Задача 10

- 1) СЕ возникает потому, что `dynamic_cast` не может работать с не полиморфными типами. Пока у нас нет виртуальных функций, типы не полиморфные.
- 2) Если раскомментировать `virtual`, все типы станут полиморфными и `dynamic_cast` проблемы не вызовет отработает в обоих случаях (`side-cast` и `downcast` соответственно, оба успешно выполнимы). Однако все равно будет СЕ в строке 2 из-за того, что обращение к `foo` у сына вызывает неоднозначность.
- 3) Если вместо `dynamic_cast` написать `static_cast`, то каст от мамы к папе будет давать СЕ, поскольку эти типы не лежат на одной линии наследования, а `static_cast` не умеет в `side-cast`. Каст же от мамы к сыну нормально отработал бы, ошибки он не вызовет. Но по-прежнему будет СЕ в строке 2 из-за того, что обращение к `foo` у сына вызывает неоднозначность.

Задача 11

В объекте `Derived`, поскольку он полиморфный, сначала лежит указатель на `vtable`, и это занимает первые 8 байт. Затем лежит два инта (`a` и `b`) по 4 байта каждый, затем еще два инта (`a` и `c`) по 4 байта каждый. Итого 24.

В чем подвох вопроса: никаких больше указателей, кроме вышеназванного на `vtable`, там лежать не должно. А переменных с именем `a` должно быть две штуки, а не одна.

Задача 12

Тут мы имеем дело с тремя версиями перегрузки, а не со специализацией. Будет 2313.

В первом случае 2, т.к. `f(int, int)` подходит лучше остальных по правилу “частное предпочтительнее общего”.

Во втором случае 1, т.к. `f(T, T)` предпочтительнее `f(T, U)` по правилу “частное лучше общего”, и она же лучше `f(int, int)`, т.к. обеспечивает идеальное соответствие, а там бы пришлось каст делать.

В третьем случае лишь версия `f(T, U)` дает идеальное соответствие, в остальных местах пришлось бы делать каст.

В 4м случае все как в первом, но мы явно указали шаблонный параметр, поэтому нешаблонная версия отпадает. Значит, будет выбрана `f(T, T)`, она лучше `f(T, U)` как более частная версия.

Задача 13

Тут я накосячил - надо было написать `Stack<int, std::vector> s;`

а я написал `Stack<int, std::vector<int>> s;`

так что никакой СЕ на самом деле не будет. Эту задачу я уже пообещал не учитывать.

Но также я сказал, что можно дать бонусные баллы тем, кто догадается, что подразумевалось. А подразумевалось использовать `template template parameters`:

```
template <typename T, template <typename> Container>
struct Stack {
    Container<T> c;
};
```

Задача 14

По стандарту компилятор парсит строку `S<T>::x * a;` как выражение, а не как объявление. Он не может парсить ее по-разному в зависимости от `T`, так как он должен еще до шаблонной подстановки уметь проверять корректность синтаксиса, в частности, отличать объявления от выражений. Поэтому возникает ошибка “`a` - необъявленная переменная”. Можно исправить ошибку двумя способами: 1) дописать `typename` перед этой строкой, и тогда он будет парсить это как объявление; 2) объявить переменную `a` где-то перед этой строкой, тогда он успешно распарсит это как выражение.

Задача 15

Ну например так.

Главное, на что смотреть: случай с нулем аргументов корректно обрабатывается, а также многоточия стоят везде где надо (и не стоят где не надо).

```
void print() {}
```

```
template <typename Head, typename... Tail>  
void print(const Head& head, const Tail&... tail) {  
    std::cout << head << '\n';  
    print(tail...);  
}
```